

# Head-to-head Evaluation of Translation-First vs Agentic Generate→Validate→Repair Pipelines for Intent-Driven NetOps Changes — Validator-Acceptance Parity under Shared-Candidate Semantics

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We report a fully preregistered, artifact-archived paired experiment (CAND-2026-001-prereg-v1) that compared two operational architectures for intent→configuration NetOps changes across heterogeneous vendor CLIs and Mininet-derived topologies: (A) translation-first (constrained vendor DSL → formal verifier → solver) and (B) agentic generate→validate→repair (hosted LLM + deterministic validators). The run declared gpt-5-mini for generation and deterministic\_netops\_validator\_v5 for deterministic end-to-end correctness checks; preregistration used shared\_initial\_candidate semantics so each task pair received a single LLM candidate shared between arms. Execution completed 240 episodes (120 paired tasks) with model\_calls\_used = 120 (one shared generation per pair). Deterministic scoring produced contingency counts n11 = 120, n10 = 0, n01 = 0, n00 = 0 (baseline = 120/120; guarded = 120/120). Paired difference (A - B) = 0.00; paired-bootstrap 95% CI = [0.00, 0.00] (10,000 resamples, seed 1729); exact McNemar two-sided p = 1.0. Because generation was intentionally shared, these outcomes measure validator-acceptance parity for identical candidates rather than independent generator or repair robustness. We provide artifact-grounded evidence (execution and analysis manifests, per-task validator traces), an explicit evidence-to-claim mapping table, a nearest-work comparison matrix, and preregistered protocol modifications required to obtain a discriminating head-to-head comparison in follow-ups.

## I. INTRODUCTION

Automating intent→configuration translation and safe sequencing of multi-vendor NetOps changes is operationally critical: production networks require both correctness guarantees (reachability, isolation, absence of transient safety violations) and flexibility to express diverse intents across vendor CLIs. Two architectural approaches are common: translation-first pipelines that restrict generation to a vendor DSL and apply formal verification/solvers, and agentic generate→validate→repair pipelines that centralize generation in an LLM and rely on deterministic validators plus bounded repair. Prior literature emphasizes both deterministic guarantees (formal verification) and the promise of LLM-driven flexibility, but direct, preregistered comparisons that produce artifactized per-task validator traces are rare.

This study’s preregistered head-to-head question was: for intent-driven network changes across heterogeneous vendor

CLIs and realistic emulator topologies, which architecture yields higher end-to-end operational correctness (measured by deterministic validation: reachability and policy isolation) — (A) translation-first or (B) agentic generate→validate→repair? The preregistration (CAND-2026-001-prereg-v1) fixed a paired design (N = 120 paired tasks), the primary estimand (paired difference in binary end-to-end correctness A - B), the generator (gpt-5-mini), the deterministic validator (deterministic\_netops\_validator\_v5), and the analysis executor (paired\_binary\_analysis\_v1).

A decisive preregistered design choice was shared\_initial\_candidate semantics: each paired task used a single LLM candidate generated once and supplied identically to both architectures. This choice deliberately removes generator variance from the confirmatory estimand, turning the experiment into a validator-acceptance parity test for identical candidates rather than an independent generator or repair efficacy comparison. The remainder of the paper (Methods, Results, Discussion) makes that restriction explicit and maps preregistered hypotheses to the measured estimand with artifact references.

## II. RELATED WORK

Two research traditions frame our contribution. Formal translation→verification systems provide soundness and solver-assisted synthesis for network configurations; they emphasize deterministic guarantees and incremental verification [10,11,12]. LLM-driven NetOps and agentic AIOps work investigates flexible generation and iterative repair, and stresses the role of deterministic validators to bound hallucinations and unsafe proposals [3,4,5]. Representative nearest works include INTA (ICNP 2025) that evaluates intent translation strategies and LLM agents [3], Mahmood & Song (IFIP Networking 2026) showing self-correcting multi-agent verifier pipelines [4], and Weighted NetKAT/formal verification literature that underpins translation-first guarantees [5,10–12].

Nearest-work comparison (compact). Table 1 (in manuscript) summarizes four experimental axes—independent generation sampling, repair-loop instrumentation, validator

acceptance focus, and emulator/hardware realism—across selected prior work and this run. The present run uniquely combines preregistration, archived paired traces, and deterministic per-task validator artifacts under shared\_initial\_candidate semantics. Unlike prior independent-generation experiments, this run isolates downstream orchestration and validation behavior given identical candidates.

### III. METHODOLOGY

Preregistration and execution contract. The preregistration (CAND-2026-001-prereg-v1) fixed the primary estimand (paired difference in binary end-to-end correctness  $A - B$ ),  $N_{\text{confirmatory}} = 120$  paired tasks, generator = gpt-5-mini, validator = deterministic\_netops\_validator\_v5 (v5.0), adapter\_family = hosted\_netops\_gvr\_v1, maximum\_model\_calls = 240, and shared\_initial\_candidate semantics. The preregistration also specified paired\_binary\_analysis\_v1 (bootstrap percentile CIs and exact McNemar test) and a power plan targeting a  $\sim 0.15$  paired difference with  $N=120$ . The preregistration artifact is archived and its checksum is recorded in the execution manifest.

Task generation and difficulty calibration. Confirmatory tasks (120 pairs) were generated by deterministic\_netops\_task\_generator\_v5. Each manifest entry (execution/task\_manifest.jsonl) contains: intent text, canonical reference answer, initial and expected final states, required safe sequences, and difficulty metadata (changed\_interface\_count, dependency\_rule\_count, level: medium/hard/very\_hard). Representative entries (e.g., task-000001) and their canonical reference answers are archived and used by the deterministic validator as ground truth for normalized comparison.

Generation and shared-candidate semantics. The orchestration implemented shared\_initial\_candidate: for each pair the system made a single LLM call and distributed the identical textual candidate to both downstream arms. Execution accounting (execution/execution\_manifest.json) records planned\_episode\_count = 240, completed\_episode\_count = 240, and model\_calls\_used = 120 (one shared generation per pair). Provider audit (analysis/deterministic\_reconciliation.json) reports hosted\_model\_call\_event\_count = 120 and cross\_condition\_cache\_key\_reuse\_observed = false, consistent with execution isolation.

Validator rules and scoring. The deterministic\_netops\_validator\_v5 applies full\_intent\_constraint\_validation: syntactic normalization, required command inclusion, ordering/dependency checks (make-before-break, shutdown→modify→restore), final state consistency, and emits validation\_after.valid (boolean). A configuration is scored success (1) only when validation\_after.valid == true. Per-task scoring JSONs (execution/scoring/task-000XXX-\*.json) record normalized\_response, parsed\_command\_count, required\_command\_count, validation\_before/after snapshots,

repair\_applied flags and the scorer id/version. These files are the canonical evidence for every per-task decision and are archived in the execution bundle.

Analysis plan and exact procedures. Analysis followed paired\_binary\_analysis\_v1, computing contingency counts ( $n_{11}, n_{10}, n_{01}, n_{00}$ ), paired difference  $A - B$ , bootstrap percentile 95% CI (10,000 resamples, bootstrap\_seed = 1729), and exact McNemar two-sided test. Analysis artifacts include analysis/paired\_contingency\_table.csv, analysis/condition\_summary.csv, analysis/analysis\_log.jsonl (bootstrap parameters), and analysis/deterministic\_reconciliation.json. Secondary estimands (repair coverage, mean repair iterations) were preregistered; their measurement depends on nonzero repair\_applied flags in scoring JSONs.

### IV. RESULTS

Execution accounting and primary numeric outcomes. The execution manifest reports planned\_episode\_count = 240, completed\_episode\_count = 240, failed\_episode\_count = 0 and model\_calls\_used = 120 (one shared generation per paired task). Provider audit and deterministic reconciliation confirm hosted\_model\_call\_event\_count = 120, cache\_miss\_count = 120, cross\_condition\_cache\_key\_reuse\_observed = false, and complete\_pair\_count = 120, consistent with the preregistered shared\_initial\_candidate execution semantics.

Deterministic scoring and contingency counts. The complete paired contingency is  $n_{11} = 120$ ,  $n_{10} = 0$ ,  $n_{01} = 0$ ,  $n_{00} = 0$ . Marginal success rates are baseline =  $120/120 = 1.0$  and guarded =  $120/120 = 1.0$  (analysis/condition\_summary.csv records these counts explicitly). The paired difference ( $A - B$ ) equals 0.00. The paired bootstrap percentile 95% CI computed with 10,000 resamples and bootstrap\_seed = 1729 is [0.00, 0.00]; the exact McNemar two-sided p-value is 1.0. These numbers are direct outputs of the preregistered paired\_binary\_analysis\_v1 executor and are archived in analysis/paired\_contingency\_table.csv and analysis/analysis\_log.jsonl.

Repair instrumentation and secondary estimands. Across all 240 episodes the archived per-task scoring JSONs show repair\_applied = false (aggregated repair\_applied count = 0). Consequently, preregistered secondary estimands concerning repair coverage and mean repair iterations are unobserved in this execution. The absence of repair events is reflected uniformly in execution/scoring/\*.json where validator\_trace.repair\_applied = false and repair\_provider\_trace\_path is null.

Representative per-task diagnostics. Representative artifacts illustrate the validator acceptance logic and task difficulty span. Example highlights from archived scoring JSONs:

- pair-000001 (task-000001): difficulty.level = "medium"; parsed\_command\_count = 4, required\_command\_count = 4. The recorded normalized\_reference\_answer differs in command ordering from the normalized\_response (vlan and mtu lines swapped), yet validation\_after.valid = true — demonstrating that the validator’s normalization and

dependency checks accept ordering variants when required commands and final state constraints are satisfied. • pair-000002 (task-000002): difficulty.level = "hard"; parsed\_command\_count = 2, required\_command\_count = 2; normalized\_response matches the reference exactly and validation\_after.valid = true. • pair-000003 (task-000003): difficulty.level = "hard"; parsed\_command\_count = 6, required\_command\_count = 6; again validation\_after.valid = true. These representative traces show tasks across medium→hard levels, with parsed\_command\_count and required\_command\_count recorded per task and used by the deterministic validator to assert correctness.

Contamination, missingness, and execution integrity diagnostics. The preregistered contamination checks produced no flags: analysis/contamination\_summary.csv is empty. Missingness diagnostics record complete\_pairs = 120 and zero failed or unscorable episodes (analysis/missingness\_summary.csv). The deterministic\_reconciliation artifact confirms episode and pair accounting consistency and that all deterministic consistency checks passed.

Interpretation of the degenerate concordance. The observed perfect concordance (all pairs accepted by both downstream arms) yields a degenerate statistical picture: the paired difference estimate is point-mass zero, the bootstrap CI collapses to [0.00,0.00], and McNemar’s test yields  $p = 1.0$ . These statistics correctly quantify uncertainty given the observed data but arise from the preregistered decision to share LLM candidates between arms. Because generator variance was removed by design, the confirmatory outcome answers the narrower question of validator-acceptance parity for identical candidates rather than the broader question of independent generator or repair superiority. This is consistent with the preregistration which specified shared\_initial\_candidate semantics and is explicitly recorded in execution/model\_configuration.json and the execution manifest.

Why repairs were not applied in this run (artifact-grounded observation). The archived per-task traces indicate that shared LLM candidates either matched the canonical normalized reference (exact match) or satisfied the validator’s normalization and dependency checks (ordering-insensitive acceptance). Representative normalized\_response and normalized\_reference\_answer fields in execution/scoring/task-000XXX-\*.json demonstrate this pattern: validator normalization reconciled minor syntactic/order variations and all required commands and final state constraints were present, so no repair loop was triggered. This operational detail explains the observed repair\_applied = false counters without invoking any unarchived assumptions.

Consequences for preregistered hypotheses and next steps. H1 (paired difference in end-to-end correctness A - B) is measured here but under the restricted estimand implied by shared\_initial\_candidate semantics (validator-acceptance parity). H2 (agentic repair coverage advantage) is unobserved because no repair events occurred. The artifact bundle and execution accounting provide explicit levers for a discrim-

inating follow-up (remove shared candidate to reintroduce generator variance, increase model\_calls\_used to produce independent per-arm generations, enable and instrument repair attempts) — each such change would produce verifiable differences in archived metrics (hosted\_model\_call\_event\_count, nonzero repair\_applied flags, increased model\_calls\_used) that would support testing the originally preregistered repair-and-generation hypotheses.

## V. DISCUSSION

What the run measures — and what it does not. Because shared\_initial\_candidate semantics were used, the preregistered confirmatory estimand reduces to validator-acceptance parity: given identical LLM candidates, do downstream orchestration and verification paths differ in deterministic validator acceptance? The observed complete concordance ( $n_{11} = 120$ ) answers this restricted question: both validators accepted the identical candidates for every paired task. The run does not exercise independent generator variance, sampling effects, or repair efficacy; therefore claims about which architecture would produce more correct initial candidates or achieve higher repair coverage when generating independently are untested.

Evidence-to-claim mapping (compact table). Table 2 (in manuscript) maps preregistered hypotheses/estimands to measured observables and archived artifacts. Example mappings: H1 (paired difference A - B) → measured estimand = validator-acceptance parity under shared\_initial\_candidate; supporting artifacts: analysis/paired\_contingency\_table.csv and analysis/condition\_summary.csv ( $n_{11}=120, n_{10}=0, n_{01}=0, n_{00}=0$ ); verdict: tested but restricted. H2 (agentic repair coverage advantage) → observed repair\_applied count = 0 in execution/scoring/\*.json; verdict: unobserved in this execution.

Operational implications and protocol guidance for discriminating comparisons. The artifact bundle documents a reproducible protocol for validator regression and orchestrator equivalence checks under fixed candidates. To test independent-generation and repair differences the preregistered protocol must be modified (and these modifications can be preregistered and audited): (i) remove shared\_initial\_candidate so each arm receives an independent LLM generation per pair (model\_calls\_used would rise from 120 to 240 for initial candidates); (ii) allow multiple sampled initial generations per arm to estimate generator variance; (iii) enable and instrument repair iterations so repair\_applied counters become informative; (iv) increase emulator fidelity or include hardware runs to reduce simulator-to-hardware contamination risk. Each modification yields verifiable changes in execution artifacts (model\_calls\_used, hosted\_model\_call\_event\_count, nonzero repair\_applied flags) and therefore supports reproducible discrimination across architectures.

Threats to validity (concise). Internal validity: shared\_candidate removes generator variance by design; this is a feature not a flaw but narrows the estimand.

Construct validity: deterministic validator acceptance operationalizes a conservative correctness definition but omits human-facing properties (explainability, maintainability). External validity: synthetic tasks and Mininet-derived emulation plus a single model (gpt-5-mini) and single validator (deterministic\_netops\_validator\_v5) limit generality. Conclusion validity: degenerate concordance (all pairs concordant) produces point mass CI and  $p = 1.0$ ; these statistics correctly describe the observed data but do not imply production-level equivalence under independent operation.

Reproducibility and audit artifacts. All primary artifacts are archived and referenced by execution/execution\_manifest.json: preregistration (CAND-2026-001-prereg-v1), execution/model\_configuration.json (declared\_model\_version = gpt-5-mini), execution/task\_manifest.jsonl, execution/responses/\*, execution/scoring/\*.json, analysis/paired\_contingency\_table.csv, analysis/condition\_summary.csv, analysis/analysis\_log.jsonl, and analysis/deterministic\_reconciliation.json. Per-task validator traces (execution/scoring/task-000XXX-\*.json) are the canonical records for each scored decision. The analysis executor parameters (bootstrap\_resamples = 10000, bootstrap\_seed = 1729) are recorded in analysis/analysis\_log.jsonl and enable deterministic recomputation of the reported CI.

## VI. LIMITATIONS

- Preregistered shared\_initial\_candidate semantics were used; this intentionally removes generator variance and prevents this run from assessing independent generation robustness differences between architectures.
- The task corpus was synthetic and executed in an emulator (Mininet-derived topologies and public vendor example grammars); emulator-to-hardware divergence limits external validity to production devices.
- A single LLM (gpt-5-mini) and a single deterministic validator (deterministic\_netops\_validator\_v5) were used; results do not imply multi-model or multi-validator generality.
- No repairs were applied in this run (repair\_applied = false in per-task traces); preregistered secondary estimands about repair coverage remain unobserved for this execution.
- The binary deterministic validator acceptance metric does not capture operator-facing properties such as explanation quality, maintainability, or human trust, which matter in production but were outside the metric used here.

## VII. CONCLUSION

We executed a preregistered, artifactized paired experiment comparing translation-first and agentic generate→validate→repair NetOps pipelines under shared\_initial\_candidate semantics. For 120 paired tasks deterministic validators accepted identical LLM candidates in both arms for every pair ( $n_{11} = 120$ ,  $n_{10} = n_{01} = n_{00} = 0$ ),

yielding paired difference 0.00 (bootstrap 95% CI [0.00, 0.00], McNemar  $p = 1.0$ ). No repairs were applied (repair\_applied = false for all episodes); preregistered hypotheses about repair coverage are therefore unobserved. The run precisely measures validator-acceptance parity under controlled shared candidates and provides a reproducible artifact bundle and explicit protocol modifications for future discriminating comparisons that exercise independent generation and repair coverage.

## DISCLOSURE STATEMENT

This research was executed autonomously after the autonomy lock under the frozen master prompt archived with the run provenance. Preregistration: CAND-2026-001-prereg-v1 (archived and used to freeze the design; preregistration\_sha256 recorded in the execution manifest). Generation used gpt-5-mini (declared\_model\_version in execution/model\_configuration.json); provider record 'openai\_responses'. The hosted adapter family was hosted\_netops\_gvr\_v1. Deterministic artifacts included deterministic\_netops\_task\_generator\_v5 and deterministic\_netops\_validator\_v5 (validator\_version 5.0). The run deliberately used the preregistered shared\_initial\_candidate generation semantics: both pipeline arms received the same initial LLM candidate per task; execution accounting shows model\_calls\_used = 120 (one shared generation per pair) while planned episodes were 240. Primary analysis used paired bootstrap percentile CIs (10,000 resamples, seed 1729) and an exact McNemar two-sided test executed by paired\_binary\_analysis\_v1. No human manually edited, rewrote, or post-processed technical content after the autonomy lock. The Research Director selected the broad topic family and constrained the initial master prompt prior to the autonomy lock; the autonomous run performed literature review, final research-question selection, preregistration, code generation, experiment execution, analysis, peer review, and manuscript drafting after the lock. Immutable reference to the initial master prompt: provenance/master\_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15). The full execution and analysis artifact bundle is archived and referenced by the execution manifest included with this submission.

## REFERENCES

- [1] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, David Walker, "Don't Mind the Gap", 2016, 10.1145/2934872.2934909
- [2] Radu Stoescu, Matei Popovici, Lorina Negreanu, Costin Raiciu, "SymNet", 2016, 10.1145/2934872.2934881
- [3] Yunze Wei, Xiaohui Xie, Tianshuo Hu, Yiwei Zuo, Xinyi Chen, Kaiwen Chi, Yong Cui, "INTA: Intent-Based Translation for Network Configuration with LLM Agents", 2025, 10.1109/icnp65844.2025.11192391
- [4] Umar Mahmood, Wang-Cheol Song, "A Self-Correcting Multi-Agent LLM Pipeline for Verified Kubernetes Network Policy", 2026, 10.23919/ifipnetworking70592.2026.11578993
- [5] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318